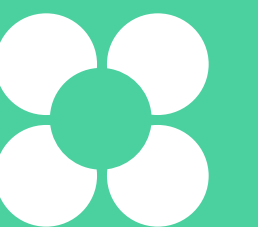


Программная работа со ссылочными типами

Обновление конфигурации

Антон Чаусов
Ведущий программист-разработчик, ITLand



Антон Чаусов

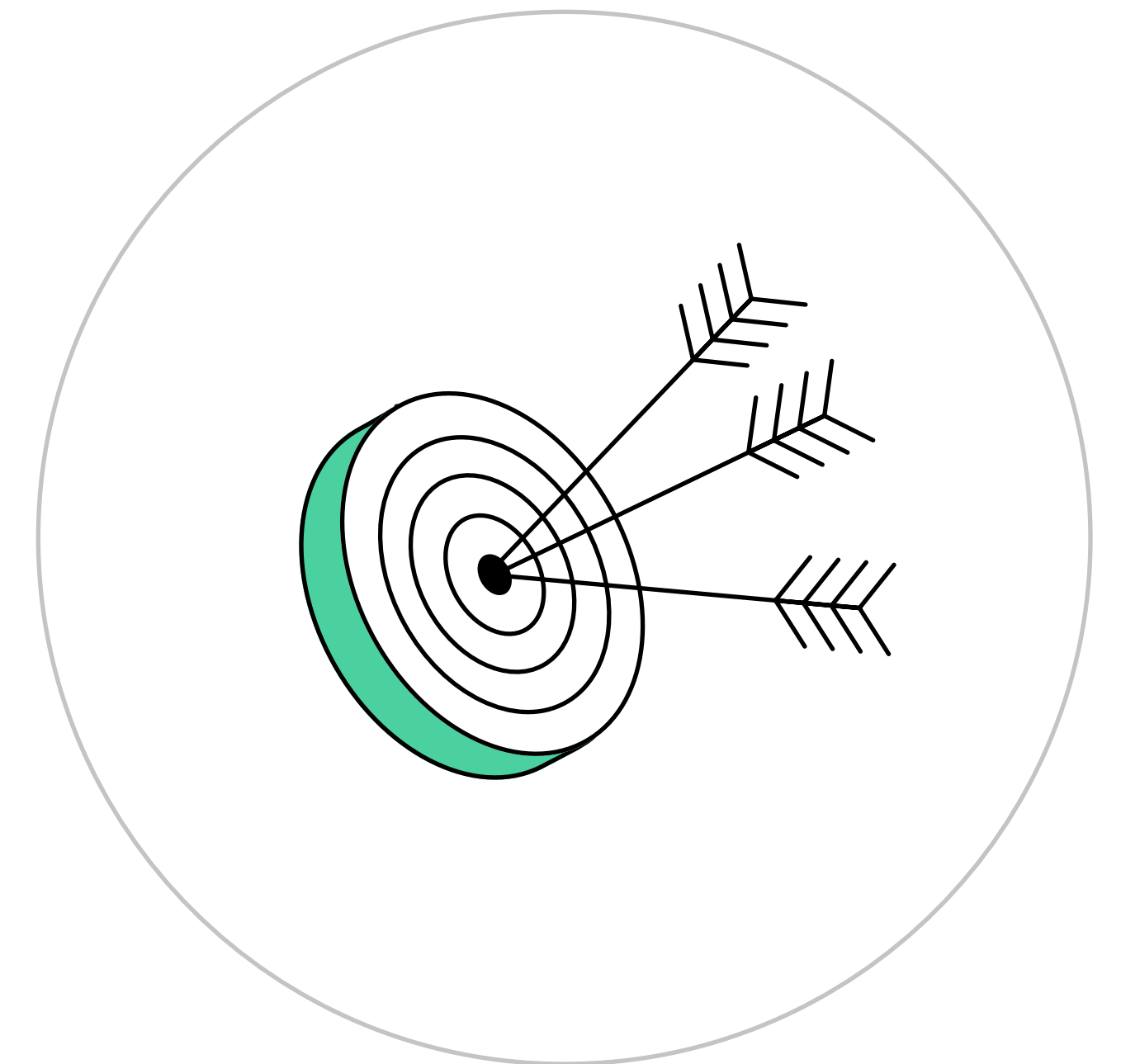
О спикере:

- ведущий программист-разработчик
- в разработке программного обеспечения с 2011 года
- 5 лет работал руководителем отдела разработки
- эксперт по технологическим вопросам фирмы 1С
- эксперт чемпионатов WorldSkills



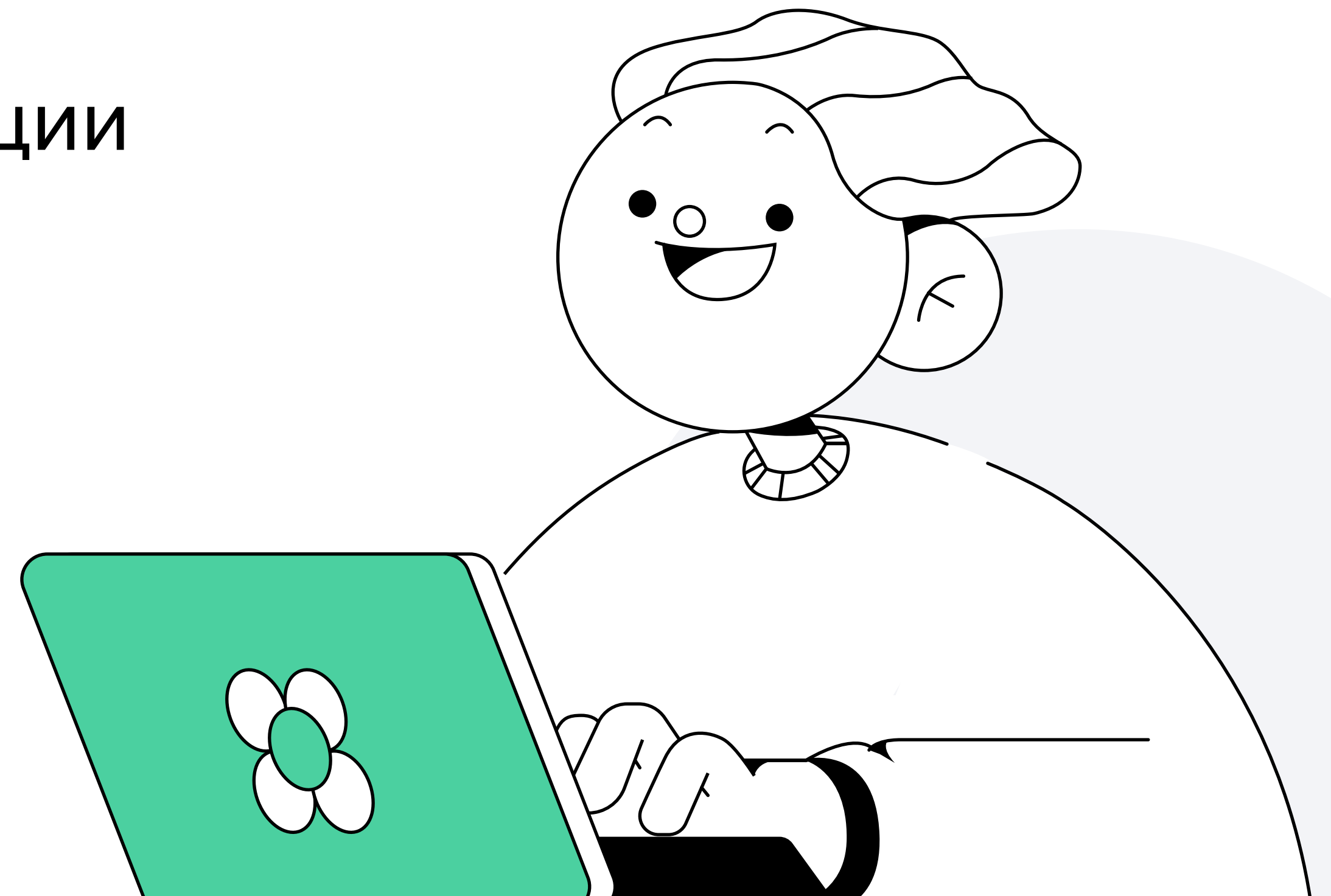
Цели занятия

- Изучить основные подходы к программной работе со справочниками и документами
- Научиться создавать, заполнять, получать список справочников и документов
- Разобрать подход к обновлению версии конфигурации 1С



План занятия

- 1 Создание и изменение справочников и документов
- 2 Выборка данных
- 3 Удаление
- 4 Обновление версии конфигурации
- 5 Домашнее задание
- 6 Итоги

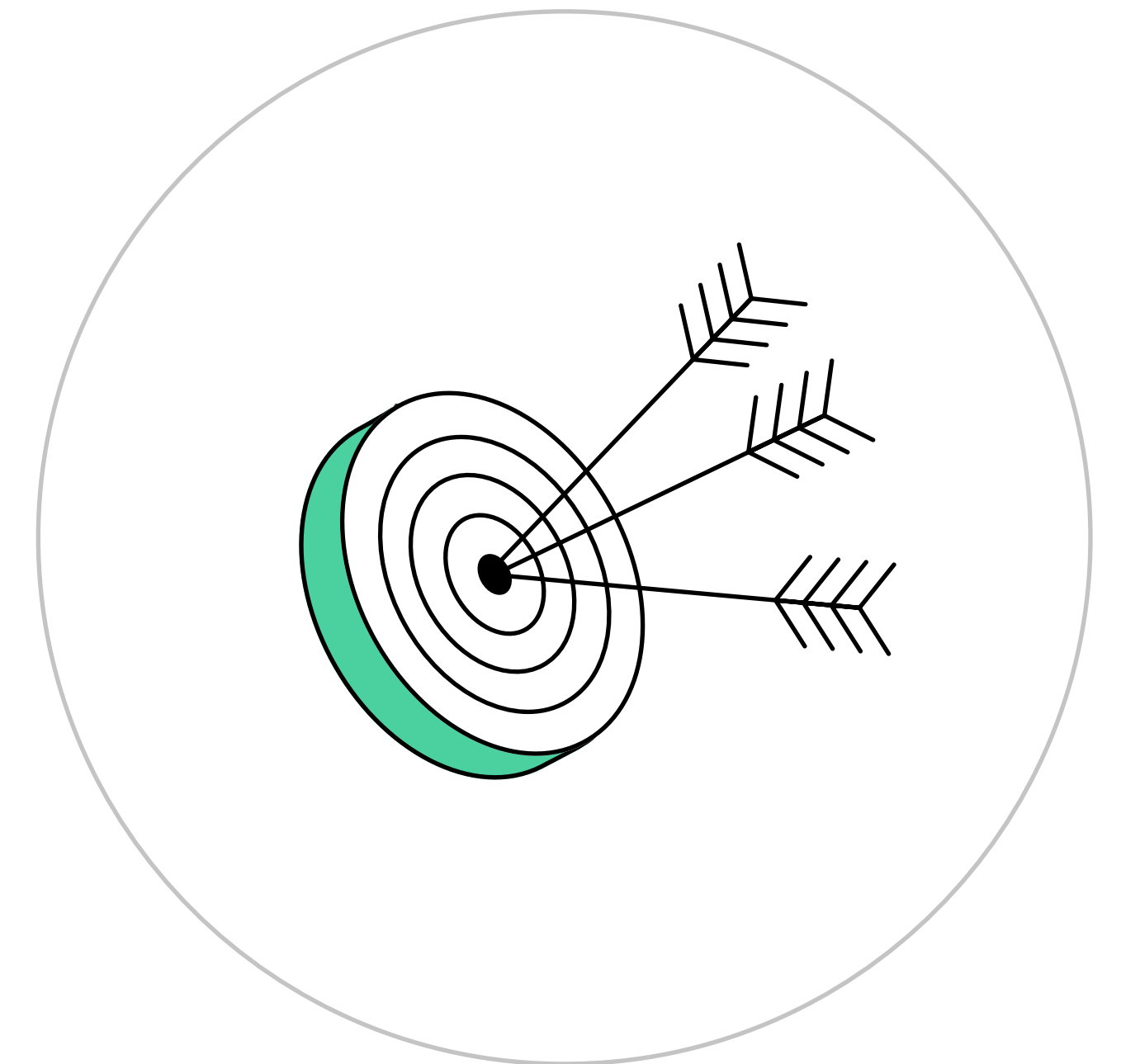


Создание и изменение справочников и документов



Цели

- Научиться программно создавать и изменять объекты ссылочных типов



Программная работа

Разработчик всегда стремится упростить работу пользователя с системой, избавить его от рутинных действий.

Часто для этого необходимо обрабатывать существующие или создавать новые справочники и документы.

С этой целью платформа 1С:Предприятие предоставляет специальные методы для каждого типа данных

Создание справочников и документов

Для создания нового элемента справочника используется метод менеджера **СоздатьЭлемент()**:

о о о

```
НовыйКонтрагент = Справочники.Контрагенты.СоздатьЭлемент();  
НовыйКонтрагент.Наименование = "ИП Ипатов А.А.";  
НовыйКонтрагент.Записать();
```

Новый документ создаётся при помощи метода **СоздатьДокумент()**:

о о о

```
НовоеСобытие = Документы.Событие.СоздатьДокумент();  
НовоеСобытие.Дата = ТекущаяДата();  
НовоеСобытие.Записать();
```

Создание справочников и документов

Если после заполнения объекта не выполнить метод **Записать()**, внесённые изменения не сохранятся в базе данных



Изменение справочников и документов

Чтобы изменить существующий объект, надо получить объект базы данных из ссылки, внести изменения и записать его:

о о о

```
&НаКлиенте
Процедура ИзменитьДокумент (Команда)
    ИзменитьДокументНаСервере (СсылкаНаДокумент) ;
КонецПроцедуры

&НаСервереБезКонтекста
Процедура ИзменитьДокументНаСервере (СсылкаНаДокумент)

    ДокументОбъект = СсылкаНаДокумент.ПолучитьОбъект () ;
    ДокументОбъект.СуммаПоДокументу = 999999 ;
    ДокументОбъект.Записать () ;

КонецПроцедуры
```

Работа с табличными частями

Работа с табличными частями похожа на работу с таблицами значений.

```
ooo
ВидЮридическийАдрес = Справочники.ВидыКонтактнойИнформации.ЮридическийАдрес;
СтрокаКонтактнойИнформации = КонтрагентОбъект.КонтактнаяИнформация.Найти (ВидЮридическийАдрес) ;
Если СтрокаКонтактнойИнформации = Неопределено Тогда
    СтрокаКонтактнойИнформации = КонтрагентОбъект.КонтактнаяИнформация.Добавить () ;
    СтрокаКонтактнойИнформации.Вид = ВидЮридическийАдрес;
КонецЕсли;
СтрокаКонтактнойИнформации.Значение = "3-я улица строителей, 25";
КонтрагентОбъект.Записать () ;
```

Табличная часть — это самостоятельный объект, который создаётся и записывается вместе с основным объектом — элементом справочника или документом

Итоги

- Программно можно создать объект информационной базы, заполнить и записать его
- Можно не только создавать новые, но и изменять существующие объекты базы данных

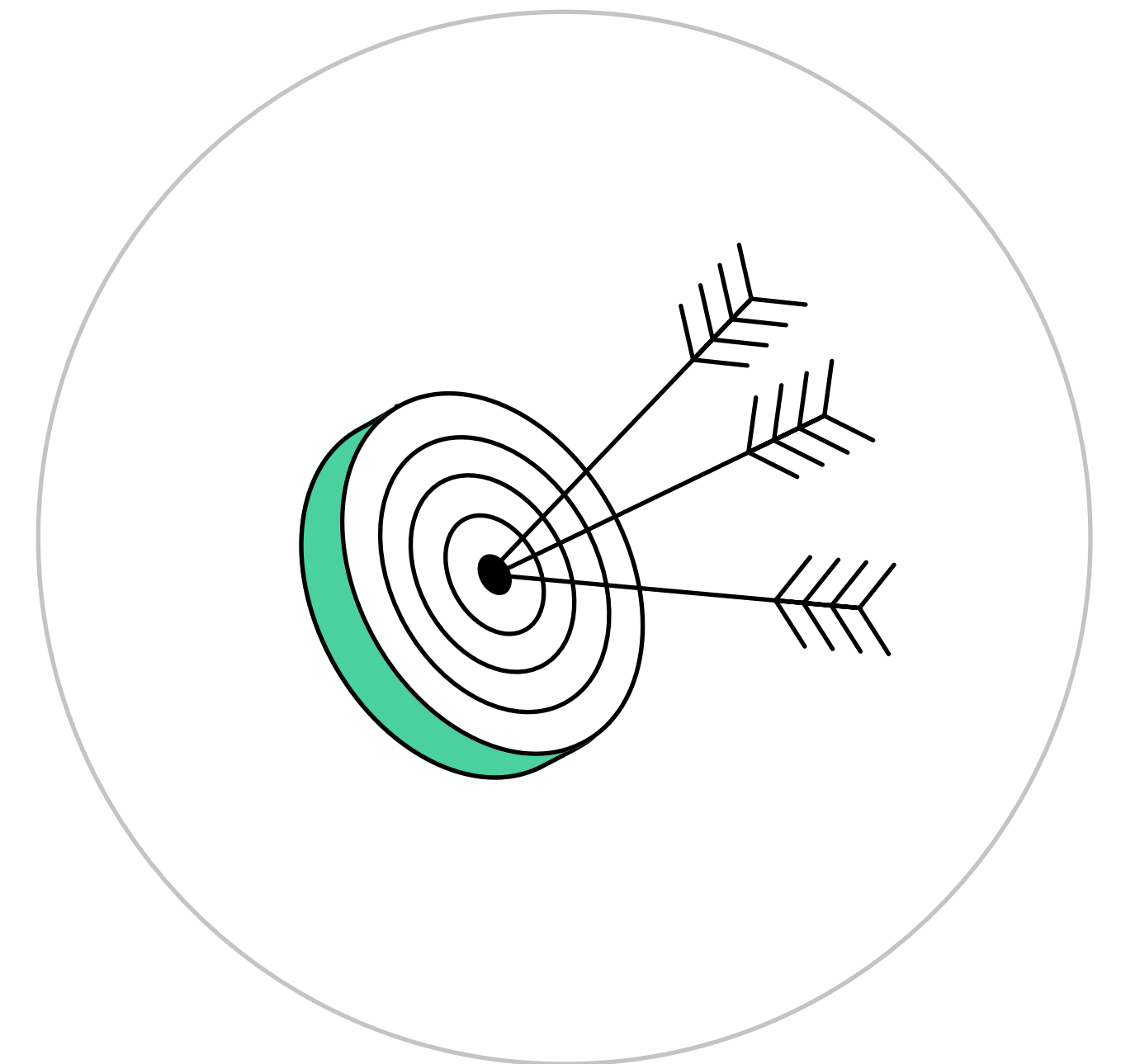


Выборка данных



Цели

- Научиться получать объекты ссылочных типов по значениям их реквизитов
- Освоить способ перебора данных объектов ссылочного типа



Поиск

Что найти один элемент, нужно использовать методы менеджера:

для Справочника:

- НайтиПоКоду
- НайтиПоНаименованию
- НайтиПоРеквизиту

для Документа

- НайтиПоНомеру
- НайтиПоРеквизиту

о о о

Функция КонтрагентСуществует (ИНН)

Контрагент = Справочники.Контрагенты.НайтиПоРеквизиту ("ИНН", ИНН);

Если Не ЗначениеЗаполнено (Контрагент) Тогда

Возврат Ложь;

КонецЕсли;

Возврат Истина;

КонецФункции

Выборка

Чтобы получить множество элементов по критерию отбора, нужно использовать метод **Выбрать()**:

```
ooo
Отбор = Новый Структура ("Оклад", 0);
Выборка = Справочники.Сотрудники.Выбрать ( , , Отбор);
Пока Выборка.Следующий() Цикл
    СотрудникОбъект = Выборка.ПолучитьОбъект();
    СотрудникОбъект.Оклад = 10000;
    СотрудникОбъект.Записать();
КонецЦикла;
```

Критерий, по которому отбираются элементы, задаётся в виде структуры.

- ❗ **Важно:** в качестве полей для отбора могут задаваться только поля «Код», «Наименование» и реквизиты справочника, для которых в конфигураторе признак индексирования установлен в значение «Индексировать» или в значение «Индексировать с доп. упорядочиванием».

Структура может содержать только один элемент

Выборка

Если надо перебрать весь справочник, критерий можно не передавать совсем:

ooo

```
ВыборкаПоВсемСотрудникам = Справочники.Сотрудники.Выбрать ( ) ;
```

Выборка

Выборка — это специальный тип объекта, при помощи которого можно перебирать справочники, документы, виды расчётов, счета, результат запроса и т. д.

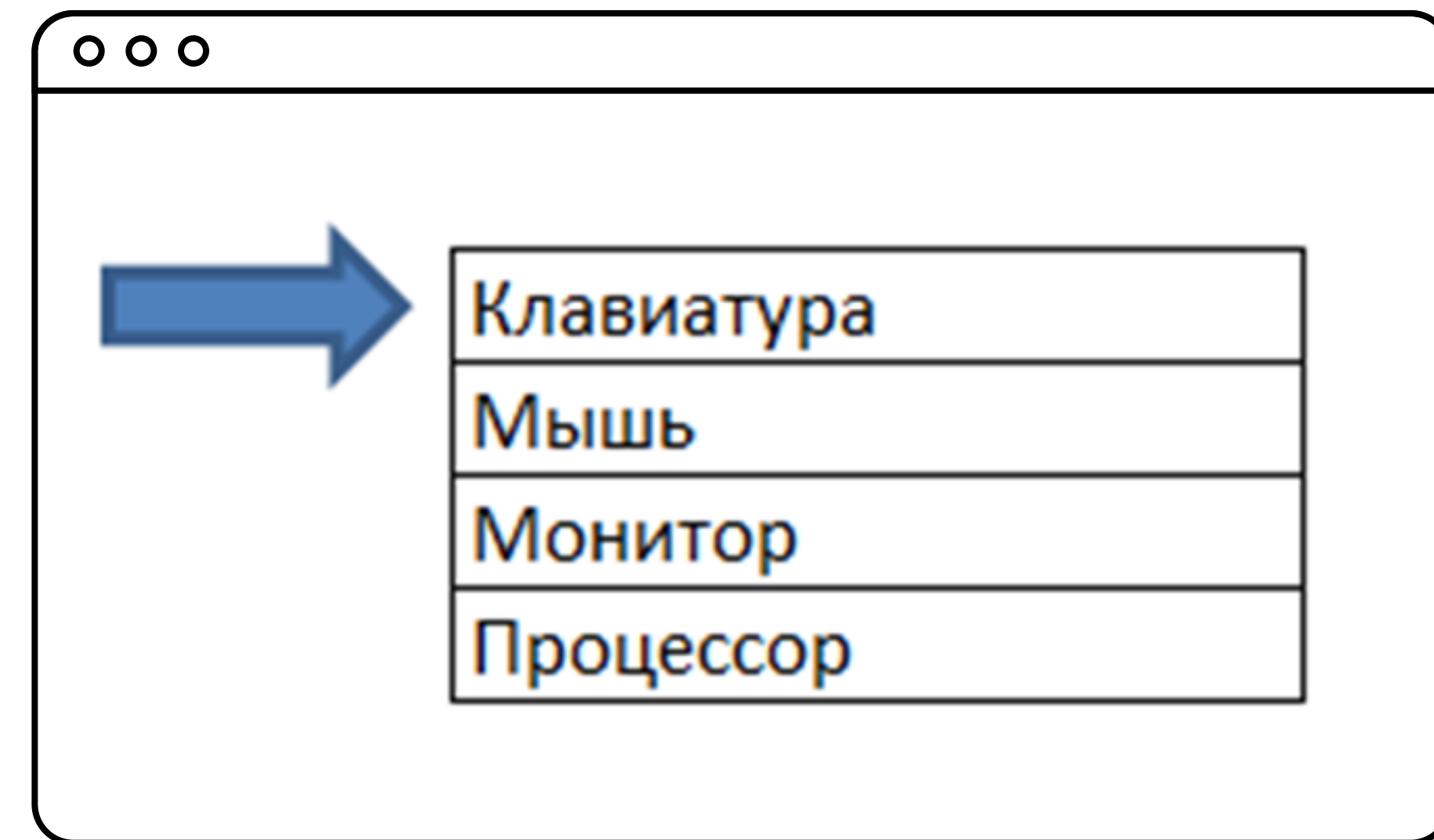
Для тех, кто знаком с работой в базах данных, выборка является аналогом курсора. Например, разработчик с помощью каких-то действий (рассмотрим их позже) получил выборку справочника и хочет её перебрать

Выборка

В самом начале выборка не будет позиционироваться ни на одном объекте:

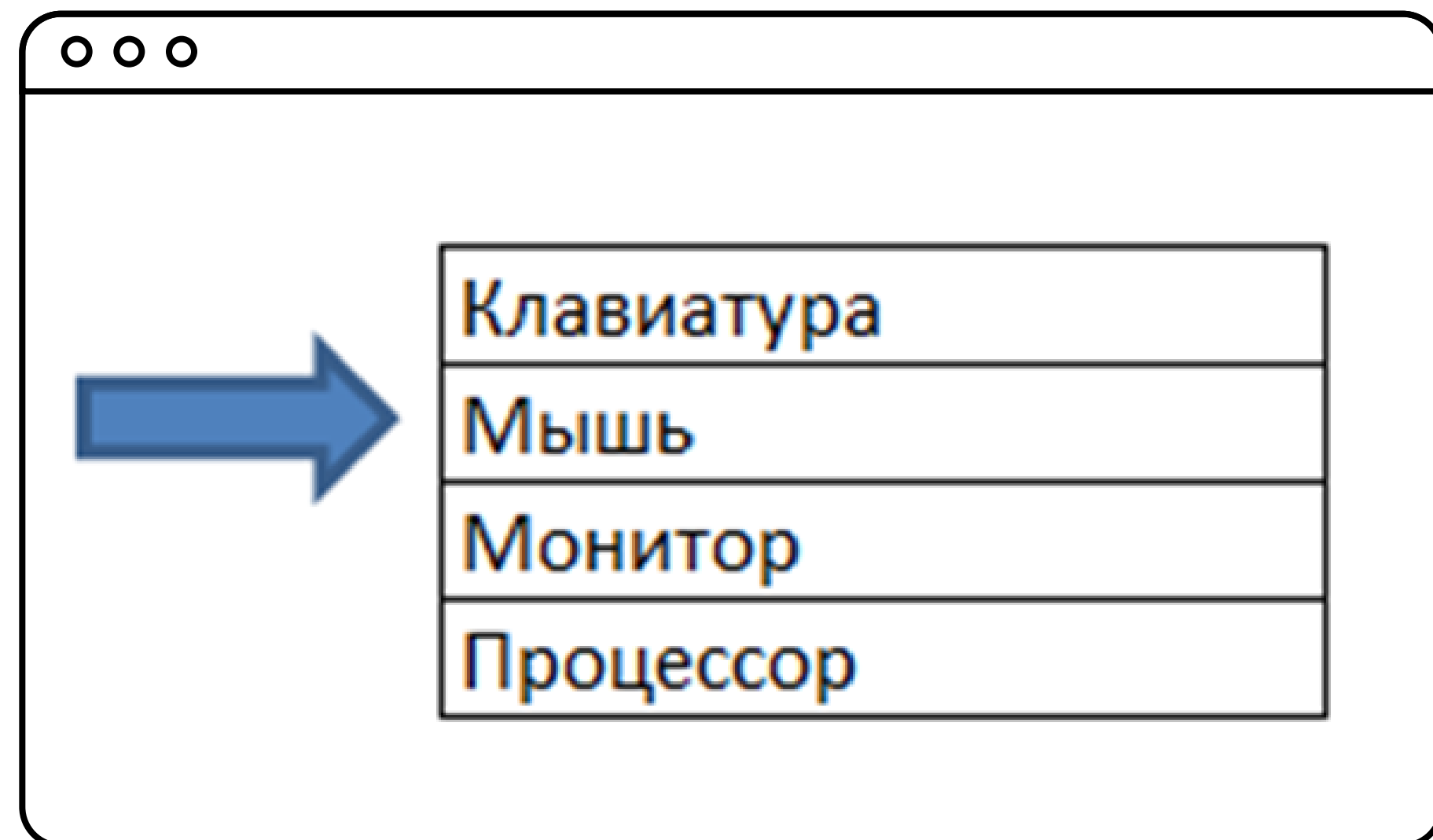


Выполнив специальный метод **Следующий()**, мы перейдём к первому элементу выборки:

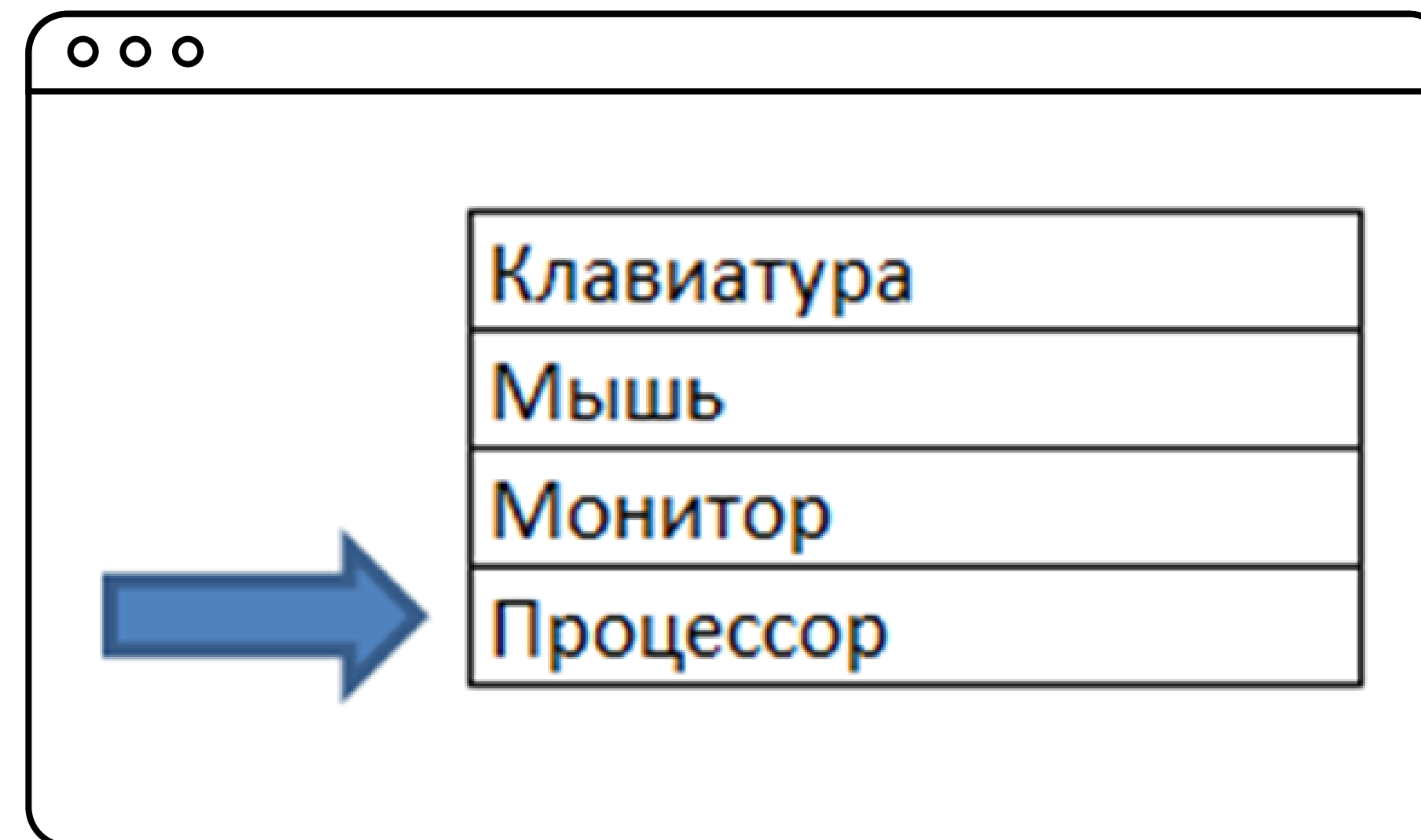


Выборка

Второе выполнение метода **Следующий()** переведёт нас на второй элемент выборки и т. д.:



Когда выборка укажет на последний элемент, метод **Следующий()** вернёт значение Ложь, так как перейти к следующему элементу будет невозможно:



Запросы

В реальных прикладных решениях почти всегда используются запросы.

Они более универсальны, позволяют задавать более сложные условия и получать данные сразу из нескольких таблиц:

о о о

```
Запрос = Новый Запрос;  
Запрос.Текст = "ВЫБРАТЬ  
|   Сотрудники.Ссылка КАК Ссылка  
| ИЗ  
|   Справочник.Сотрудники КАК Сотрудники  
| ГДЕ  
|   Сотрудники.Оклад = 0";  
  
Выборка = Запрос.Выполнить().Выбрать();  
  
Пока Выборка.Следующий() Цикл  
    СотрудникОбъект = Выборка.Ссылка.ПолучитьОбъект();  
    СотрудникОбъект.Оклад = 10000;  
    СотрудникОбъект.Записать();  
КонецЦикла;
```

Запросам и отчётам
будет посвящён отдельный
модуль курса

Итоги

- Зная номер документа, наименование/код справочника или значение реквизита, можно получить ссылку на объект базы данных
- Для обработки многих объектов можно использовать выборку
- В реальных задачах выборку формируют при помощи запроса

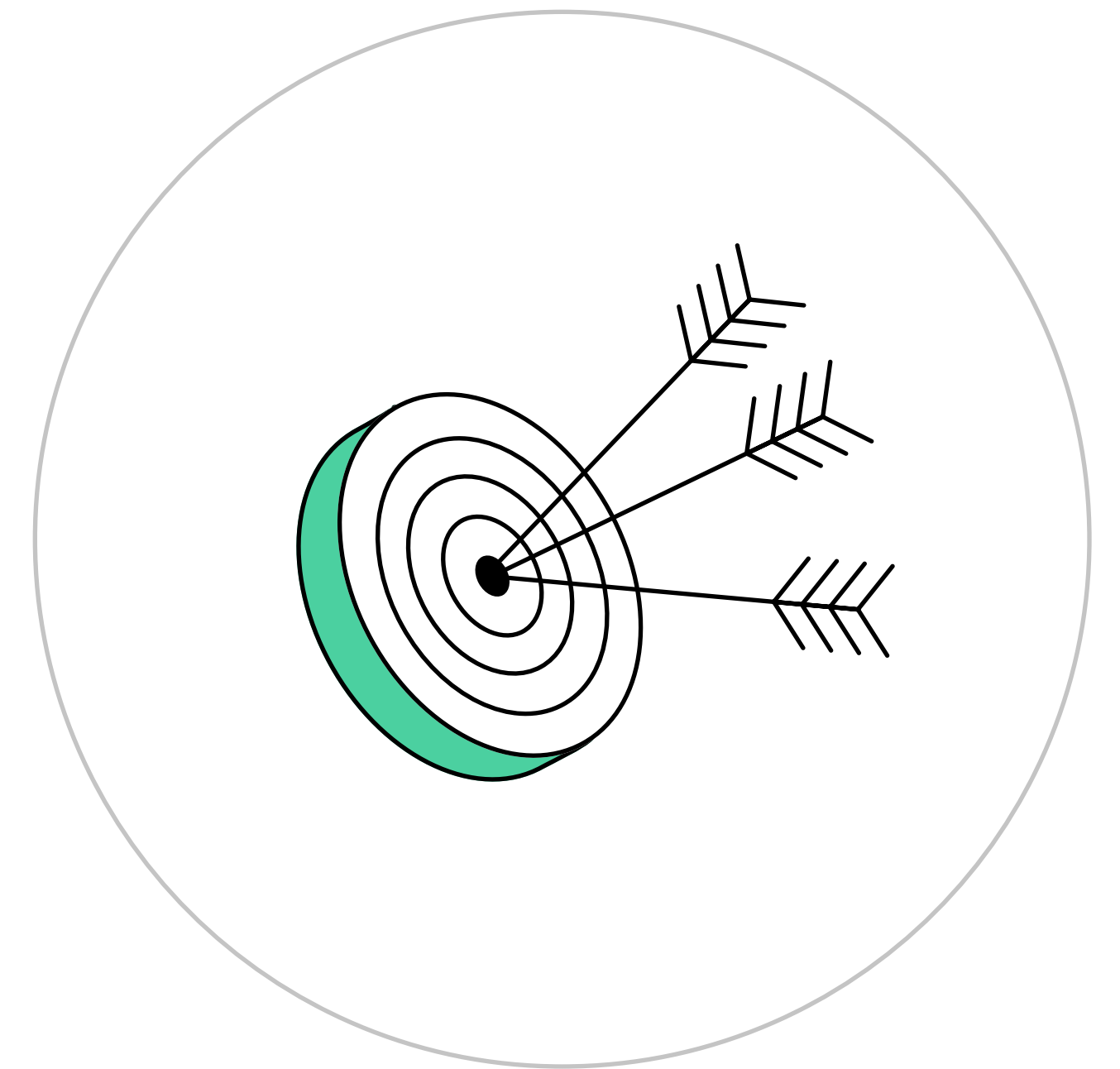


Удаление



Цели

- Научиться программно удалять объекты базы данных



Удаление

Ссылочные объекты можно удалять непосредственно в пользовательском интерфейсе. Эта возможность регулируется правом «Интерактивное удаление». Если право выдано пользователю, он может нажать Shift + Delete, после чего объект будет удалён.

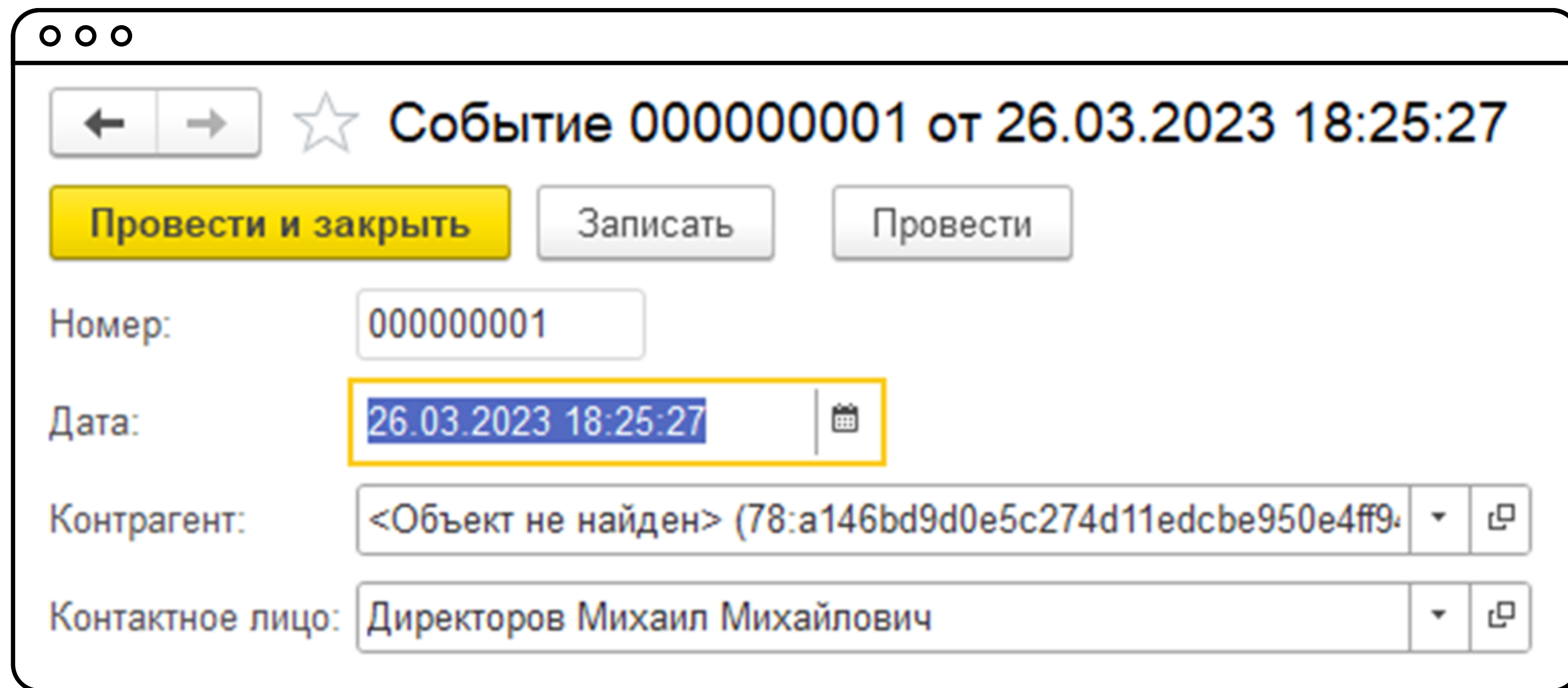
Программно то же самое можно сделать методом **Удалить()**:

о о о

```
Отбор = Новый Структура ("Оклад", 0);  
Выборка = Справочники.Сотрудники.Выбрать ( , , Отбор);  
Пока Выборка.Следующий() Цикл  
    СотрудникОбъект = Выборка.ПолучитьОбъект();  
    СотрудникОбъект.Удалить();  
КонецЦикла;
```


Удаление

Если объект удаляется таким образом, то может быть нарушена логика базы данных — ссылка на объект есть, а самого объекта нет:



The screenshot shows a web application window with a title bar containing three dots. The main content area has a header with navigation arrows, a star icon, and the text "Событие 0000000001 от 26.03.2023 18:25:27". Below the header are three buttons: "Провести и закрыть" (highlighted in yellow), "Записать", and "Провести". The form contains four fields: "Номер:" with the value "0000000001", "Дата:" with the value "26.03.2023 18:25:27" (highlighted with a yellow border), "Контрагент:" with the value "<Объект не найден> (78:a146bd9d0e5c274d11edcbe950e4ff9:" and a dropdown arrow, and "Контактное лицо:" with the value "Директоров Михаил Михайлович" and a dropdown arrow. Each dropdown menu has a small icon to its right.

Более правильный способ — пометать объекты на удаление и периодически удалять их с помощью специальной обработки

Удаление

Ненужные элементы помечаются на удаление:

- интерактивно
- установкой в Истина стандартного булевского реквизита **ПометкаУдаления**
- вызовом метода **УстановитьПометкуУдаления()**

ooo

```
Отбор = Новый Структура ("Оклад", 0);  
Выборка = Справочники.Сотрудники.Выбрать ( , , Отбор);  
Пока Выборка.Следующий() Цикл  
    СотрудникОбъект = Выборка.ПолучитьОбъект();  
    СотрудникОбъект.УстановитьПометкуУдаления(Истина);  
    СотрудникОбъект.Записать();  
КонецЦикла;
```

Удаление помеченных объектов

Периодически вызывается стандартная обработка **Удаление помеченных объектов**. Она проверяет, есть ли ссылки на помеченные элементы в других данных. Если их нет, то элементы удаляются.

Это позволяет поддерживать так называемую **ссылочную целостность**, когда любая ссылка в ИБ указывает на существующий объект.

Право «Интерактивное удаление» обычно не дают даже пользователям с полными правами

Итоги

- В системе присутствует метод для непосредственного удаления объекта из базы.

Перед использованием этого метода проверьте, что нет ссылок на удаляемый объект

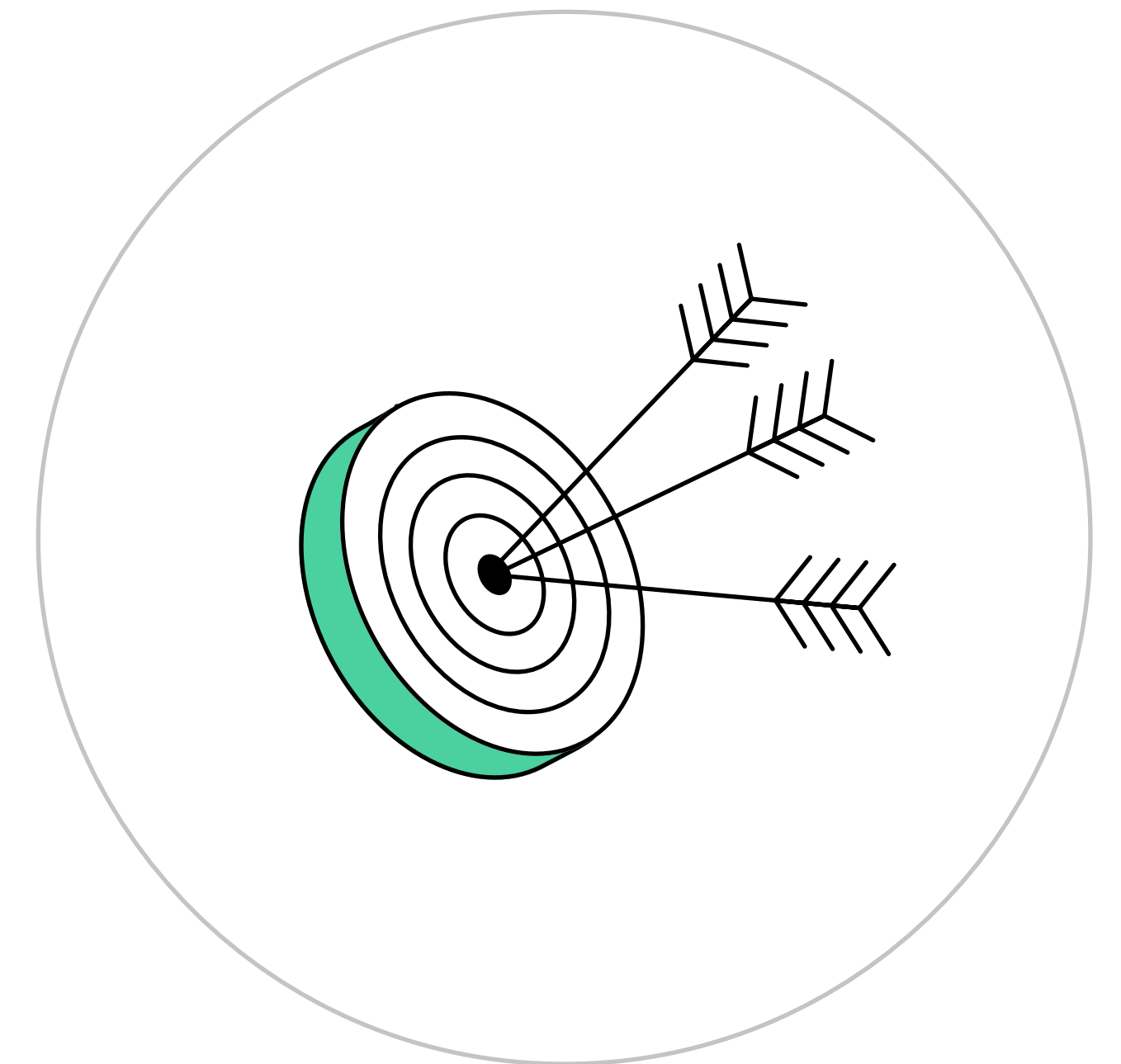


Обновление версии конфигурации



Цели

- Разобрать принцип обновления конфигурации базы данных
- На практическом примере прописать обновление конфигурации на новую версию



Обновление

Часто при смене версии конфигурации данным требуется обработка. Например, в справочник добавили новый реквизит и надо заполнить его значением по умолчанию.

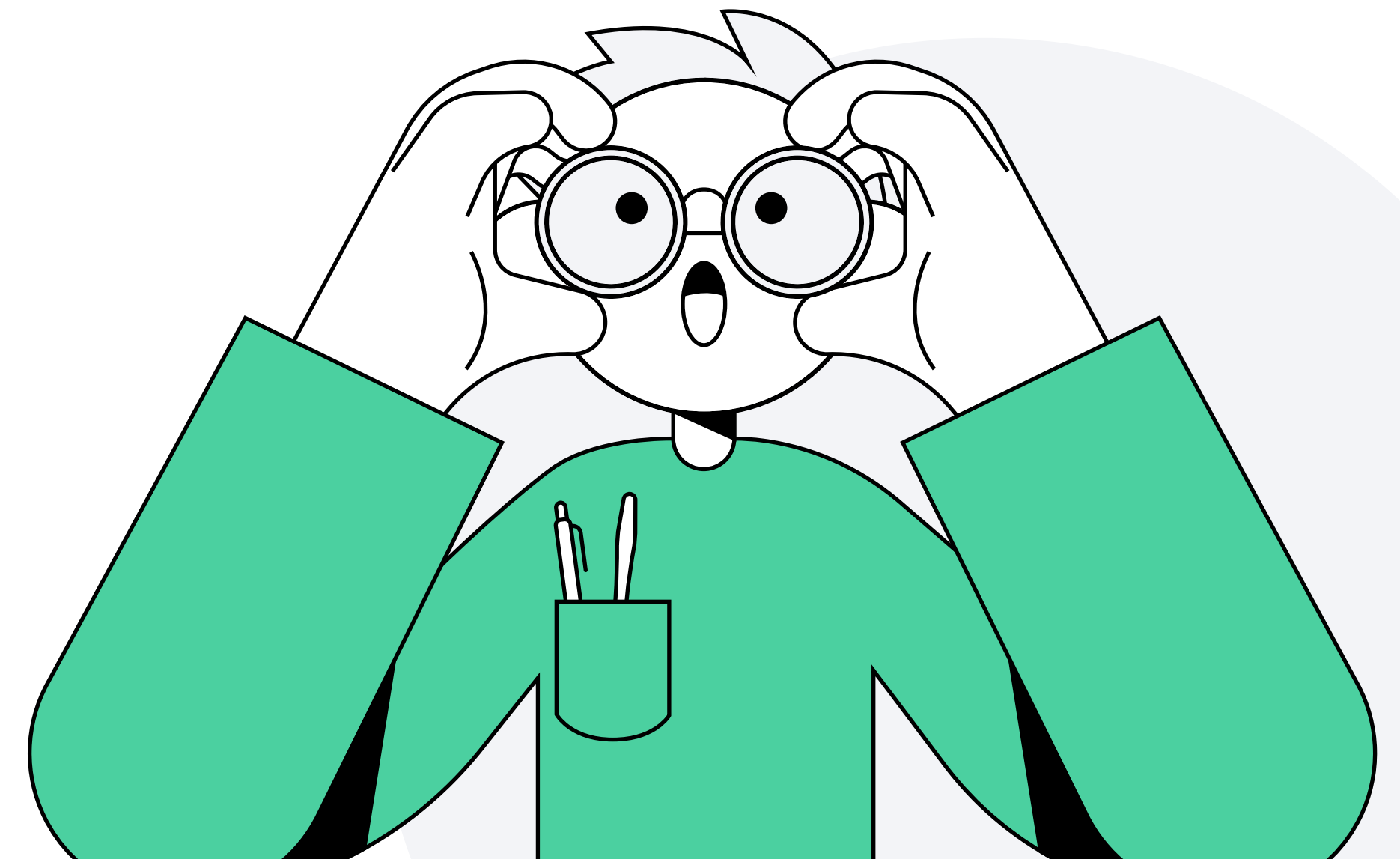
Важно организовать код так, чтобы:

- обработка данных была выполнена
- обработка данных была бы выполнена при обновлении программы до определённой (более новой) версии
- при последующих запусках конфигурации обработка не выполнялась

Обновление

Существует много подходов к реализации этой логики. Например, в типовых конфигурациях используют специальный регистр сведений **ВерсииПодсистем**.

Рассмотрим простой подход с применением константы



Обновление. Шаг 1

Убедимся, что у конфигурации заполнен номер версии

ooo

▼ Разработка:

Поставщик	Netology
Версия	1.0.0.1
Адрес каталога обновлений	

Обновление. Шаг 2

Создадим константу, которая будет содержать номер обновлённой версии.

Версия задаётся строкой, поэтому тип данных константы — также строка.
Длина строки — 15 символов, этого точно будет достаточно

The screenshot shows a dialog box titled "Свойства: ВерсияКонфигурации" (Properties: Version Configuration). It has a standard Windows-style title bar with minimize, maximize, and close buttons. Below the title bar is a toolbar with icons for sorting (A-Z, Z-A), a list view, a search icon, and a close button. A search bar labeled "Поиск (Ctrl+Alt+I)" is located below the toolbar. The main content area is divided into two sections. The first section, titled "Основные:" (Basic:), contains three text input fields: "Имя" (Name) with the value "ВерсияКонфигурации", "Синоним" (Synonym) with the value "Версия конфигурации", and "Комментарий" (Comment) which is empty. The second section contains four configuration options: "Тип" (Type) is set to "Строка" (String) in a dropdown menu; "Длина" (Length) is set to "15" in a numeric input field; "Допустимая длина" (Allowable length) is set to "Переменная" (Variable) in a dropdown menu; and "Неограниченная длина" (Unlimited length) is an unchecked checkbox.

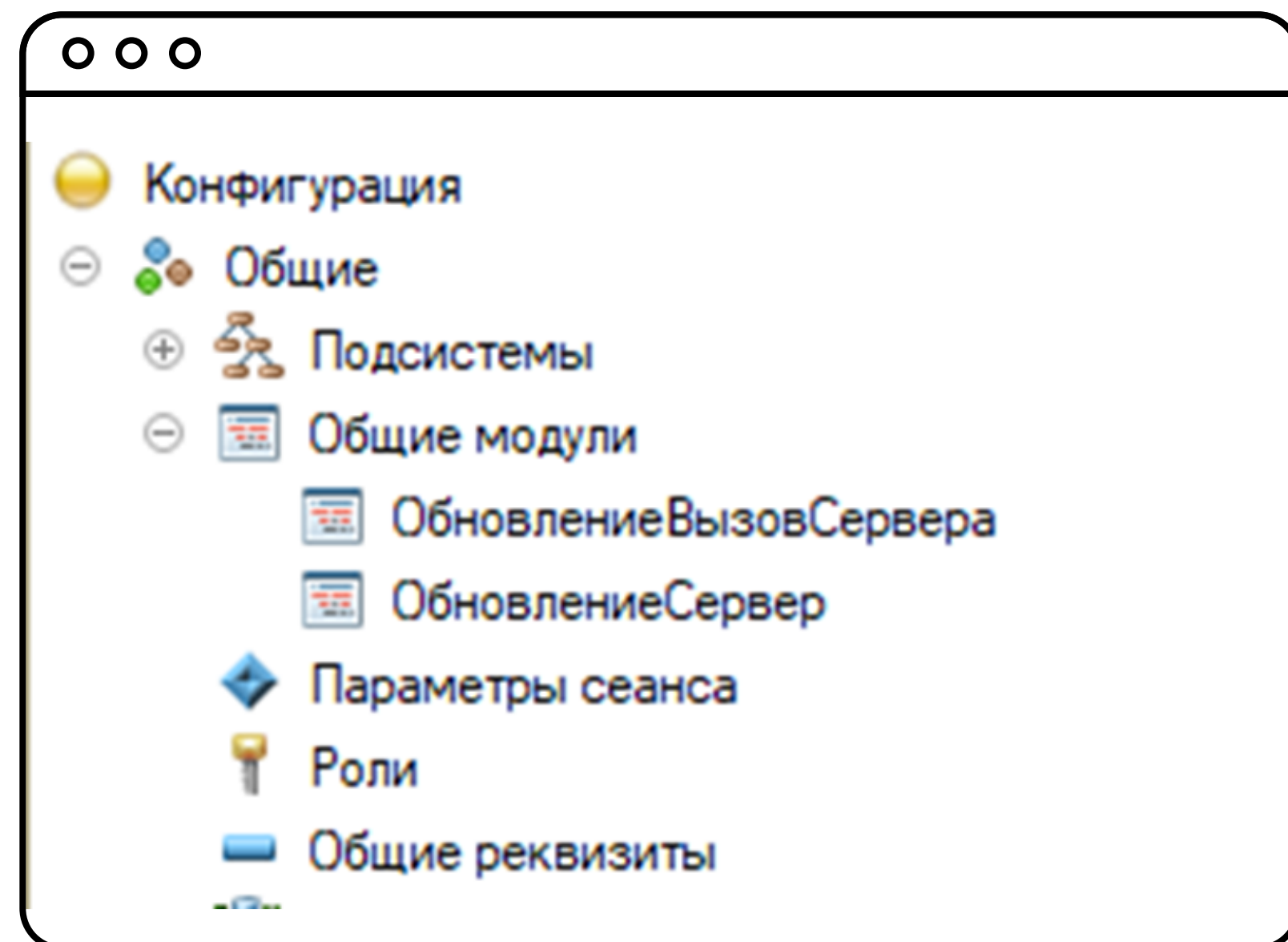
Свойства: ВерсияКонфигурации	
Поиск (Ctrl+Alt+I)	
▼ Основные:	
Имя	ВерсияКонфигурации
Синоним	Версия конфигурации
Комментарий	
Тип	Строка
Длина	15
Допустимая длина	Переменная
Неограниченная длина	☐

Обновление. Шаг 3

Выгрузим базу данных версии 1.0.0.1 для проверки обновления.

При обновлении конфигурации до версии 1.0.0.2 будем проверять всю номенклатуру в справочнике и, если не заполнен тип номенклатуры, устанавливать по умолчанию значение **Товар**.

Проверка версии должна быть запущена при начале работы системы (в модуле приложения), но работать с данными базы можно только на стороне сервера, поэтому нам понадобится общий серверный модуль



Обновление. Шаг 4

Пропишем код в модуле менеджера справочника. Поскольку мы обрабатываем всю таблицу, логично разместить процедуру именно в модуле менеджера:

о о о

Процедура ЗаполнитьТипНоменклатурыПоУмолчанию() Экспорт

```
// Мы находимся в модуле менеджера Номенклатуры,  
// поэтому тут можно не писать "Справочники.Номенклатура"  
Выборка = Выбрать();
```

```
ТипТовар = Перечисления.ТипНоменклатуры.Товар;  
ПустойТип = Перечисления.ТипНоменклатуры.ПустаяСсылка();
```

Пока Выборка.Следующий() Цикл

```
// Можно было бы сразу отобрать только номенклатуру с пустым типом,  
// но для этого пришлось бы добавить индекс в справочник.
```

```
Если Выборка.ТипНоменклатуры <> ПустойТип Тогда  
    Продолжить;  
КонецЕсли;
```

```
НоменклатураОбъект = Выборка.ПолучитьОбъект();  
НоменклатураОбъект.ТипНоменклатуры = ТипТовар;  
НоменклатураОбъект.Записать();
```

КонецЦикла;

КонецПроцедуры

Обновление. Шаг 5

Пропишем код в серверном модуле:

- ① Проверим версию в константе и версию в свойствах конфигурации:
 - если они совпадают — ничего делать не надо
 - если отличаются — начинаем процедуру обновления

И константа, и свойство конфигурации — строки. Если мы их сравним, то больше будет та строка, которая в алфавитном порядке идёт позже. Поэтому, например, 1.0.0.2 больше, чем 1.0.0.1. Пустая строка будет меньше любой заполненной

- ② Нам надо сравнить версию, до которой обновляется конфигурация, с текущей и, если это 1.0.0.2 или более новая версия, запустить обработку справочника
- ③ При этом если текущая версия больше или равна версии 1.0.0.2, выполнять обработчик не нужно
- ④ После обработки устанавливаем в константу новое значение версии

Обновление. Шаг 5

о о о

Процедура ВыполнитьОбновление() Экспорт

СтараяВерсия = Константы.ВерсияКонфигурации.Получить();
НоваяВерсия = Метаданные.Версия;

Если СтараяВерсия = НоваяВерсия Тогда
 Возврат;
КонецЕсли;

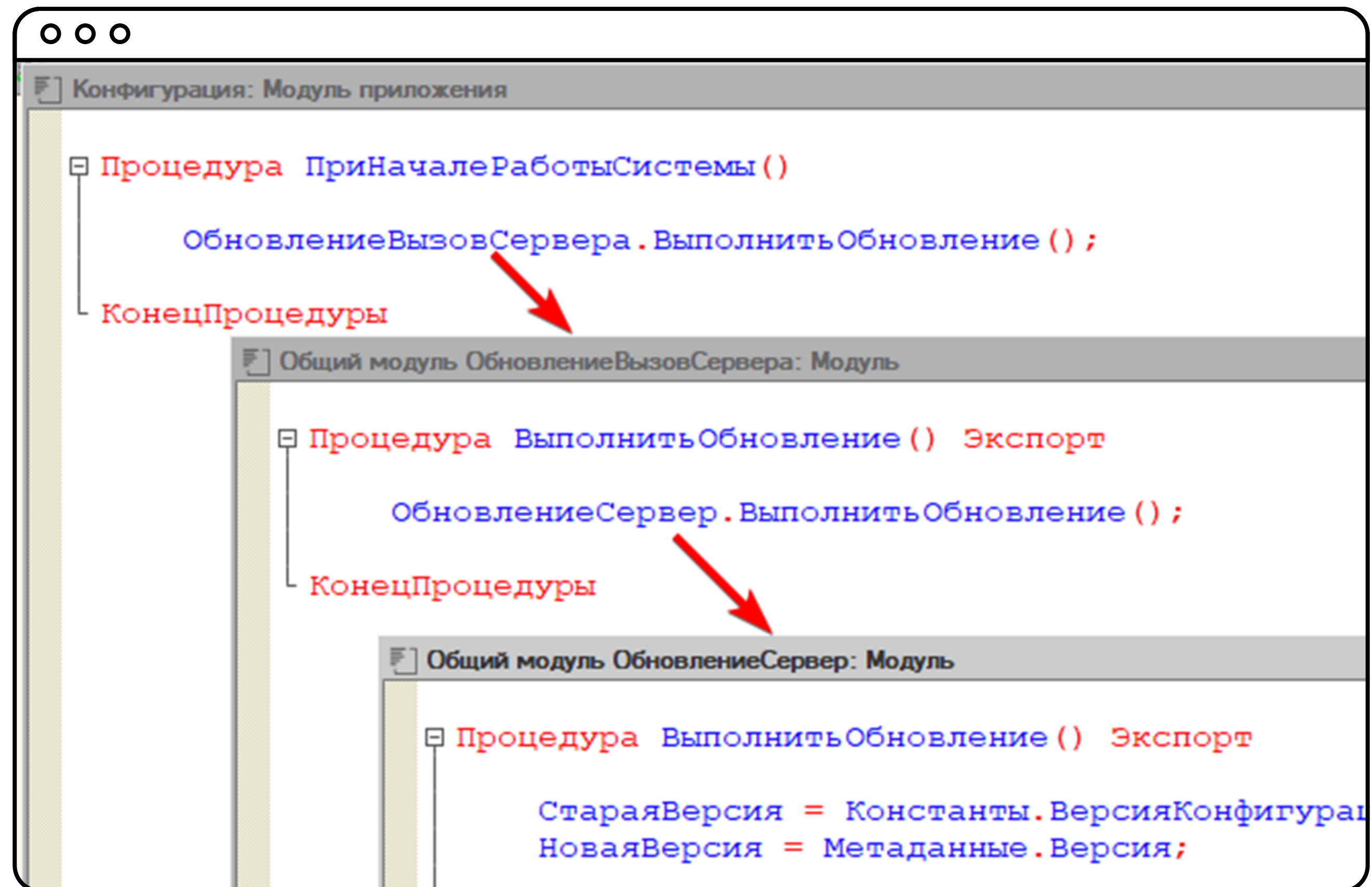
Если СтараяВерсия < "1.0.0.2" И НоваяВерсия >= "1.0.0.2" Тогда
 Справочники.Номенклатура.ЗаполнитьТипНоменклатурыПоУмолчанию();
КонецЕсли;

Константы.ВерсияКонфигурации.Установить(НоваяВерсия);

КонецПроцедуры

Обновление. Шаг 6

Осталось только прописать вызов этого метода в модуле приложения:



Обновление. Шаг 7

Для запуска обновления в свойствах конфигурации меняем версию на 1.0.0.2



Обновление. Шаг 8

Если понадобится выполнить обновление с обработкой данных на новую версию, мы просто добавим новое условие:

о о о

Процедура ВыполнитьОбновление() Экспорт

СтараяВерсия = Константы.ВерсияКонфигурации.Получить();

НоваяВерсия = Метаданные.Версия;

Если СтараяВерсия = НоваяВерсия Тогда

Возврат;

КонецЕсли;

Если СтараяВерсия < "1.0.0.2" И НоваяВерсия >= "1.0.0.2" Тогда

Справочники.Номенклатура.ЗаполнитьТипНоменклатурыПоУмолчанию();

КонецЕсли;

Если СтараяВерсия < "1.0.0.5" И НоваяВерсия >= "1.0.0.5" Тогда

// Прописать команды, которые обработают данные при переходе на версию "1.0.0.5"

КонецЕсли;

Константы.ВерсияКонфигурации.Установить(НоваяВерсия);

КонецПроцедуры

Итоги

- Обновление подразумевает переход с одной версии конфигурации на другую, при этом для хранения текущей версии конфигурации можно использовать константу. В реальных решениях чаще применяют регистр сведений



Итоги занятия

- Справочники применяются для описания объектов предметной области
- При помощи документов описываются события, которые происходят в предметной области
- Справочники и документы схожи по своей структуре, но их внутреннее поведение отличается



Спасибо за внимание!

Антон Чаусов
Ведущий программист-разработчик, ITLand

